

SQLITE

- Introduction
- Connection establishment
- Table creation and handling
- Fetch all



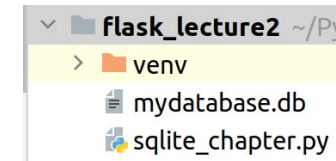
SQLite introduction

- SQLite is a lightweight, embedded relational database management system. It is written in C and provides a SQL interface for interacting with the database. SQLite is often used in mobile devices, embedded systems, and web browsers, as it does not require a separate server process and can read and write directly to ordinary disk files. It is also a popular choice for small to medium-sized applications that need a simple, reliable, and low-overhead database management system.
- SQLite can be easily integrated with Python using the sqlite3 module, which is included in the Python standard library. The sqlite3 module provides a simple and convenient way to interact with an SQLite database from within a Python script.
- To use the sqlite3 module, you first need to import it. Once imported, you can create a connection to a database using the connect() method, and create a cursor object to execute SQL commands. The cursor object can be used to execute SELECT, INSERT, UPDATE, and DELETE statements, as well as create and alter tables.

SQLITE integration with python

- To use SQLite3 in Python, first of all, you will have to import the sqlite3 module and then create a connection object which will connect us to the database and will let us execute the SQL statements.
- You can a connection object using the connect() function:

```
import sqlite3  
con = sqlite3.connect('mydatabase.db')
```



- To execute SQLite statements in Python, you need a cursor object. You can create it using the cursor() method.
- The SQLite3 cursor is a method of the connection object. To execute the SQLite3 statements, you should establish a connection at first and then create an object of the cursor using the connection object as follows:

```
con = sqlite3.connect('mydatabase.db')  
cursorObj = con.cursor()
```

SQLite integration with python

When you create a connection with SQLite, that will create a database file automatically if it doesn't already exist. This database file is created on disk; we can also create a database in RAM by using :memory: with the connect function. This database is called in-memory database.

```
import sqlite3
from sqlite3 import Error
def sql_connection():
    try:
        con = sqlite3.connect(':memory:')
        print("Connection is established: Database is created in memory")
    except Error:
        print(Error)
    finally:
        con.close()
sql_connection()
```

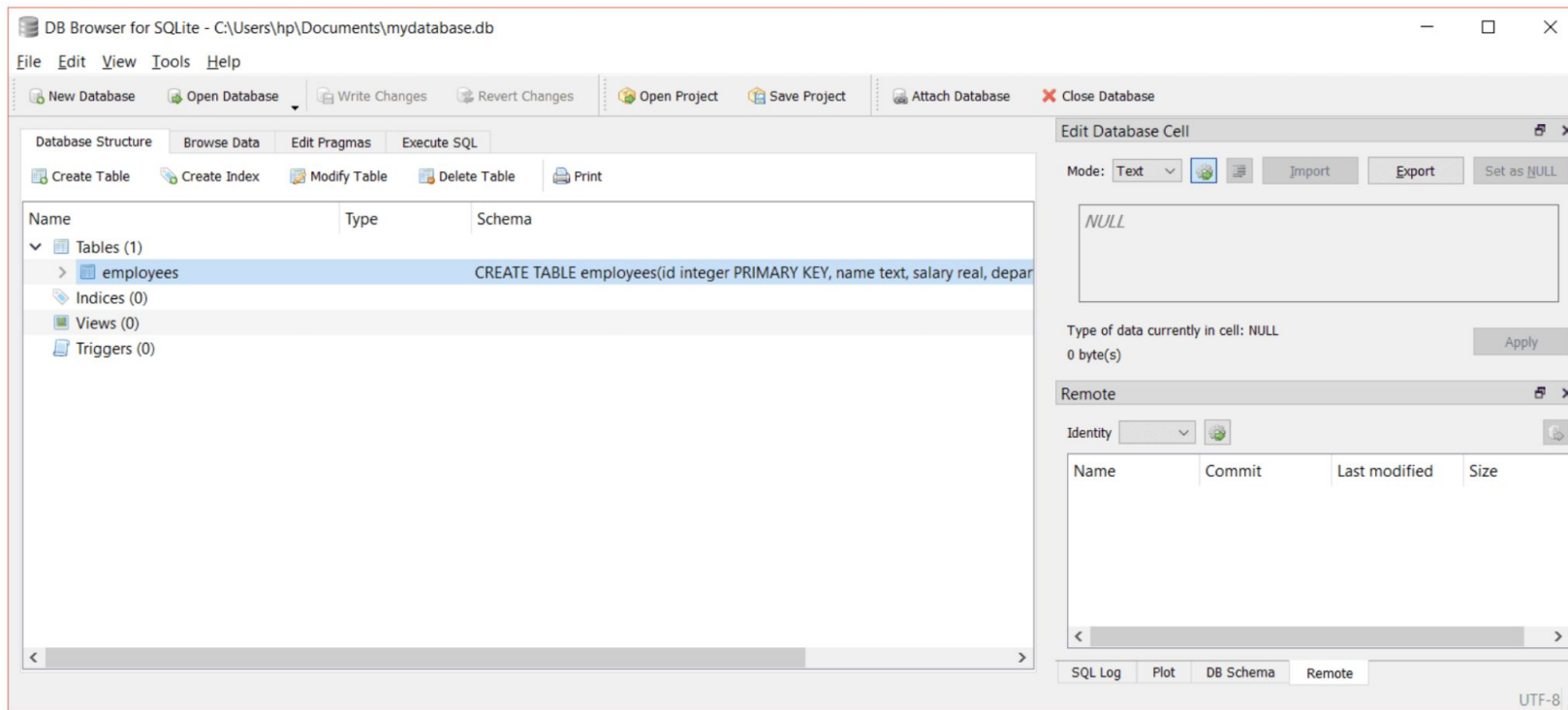
SQLITE integration with python

- To create a table in SQLite3, you can use the Create Table query in the execute() method. Consider the following steps:
 - Create a connection object.
 - From the connection object, create a cursor object.
 - Using the cursor object, call the execute method with create table query as the parameter.

```
sqlite_chapter.py
1 import sqlite3
2 from sqlite3 import Error
3 def sql_connection():
4     try:
5         con = sqlite3.connect('mydatabase.db')
6         return con
7     except Error:
8         print(Error)
9 def sql_table(con):
10     cursorObj = con.cursor()
11     cursorObj.execute("CREATE TABLE employees(id integer PRIMARY KEY, name text, salary real, department text, position text, hireDate text)")
12     con.commit()
13 con = sql_connection()
14 sql_table(con)
```

DB browser

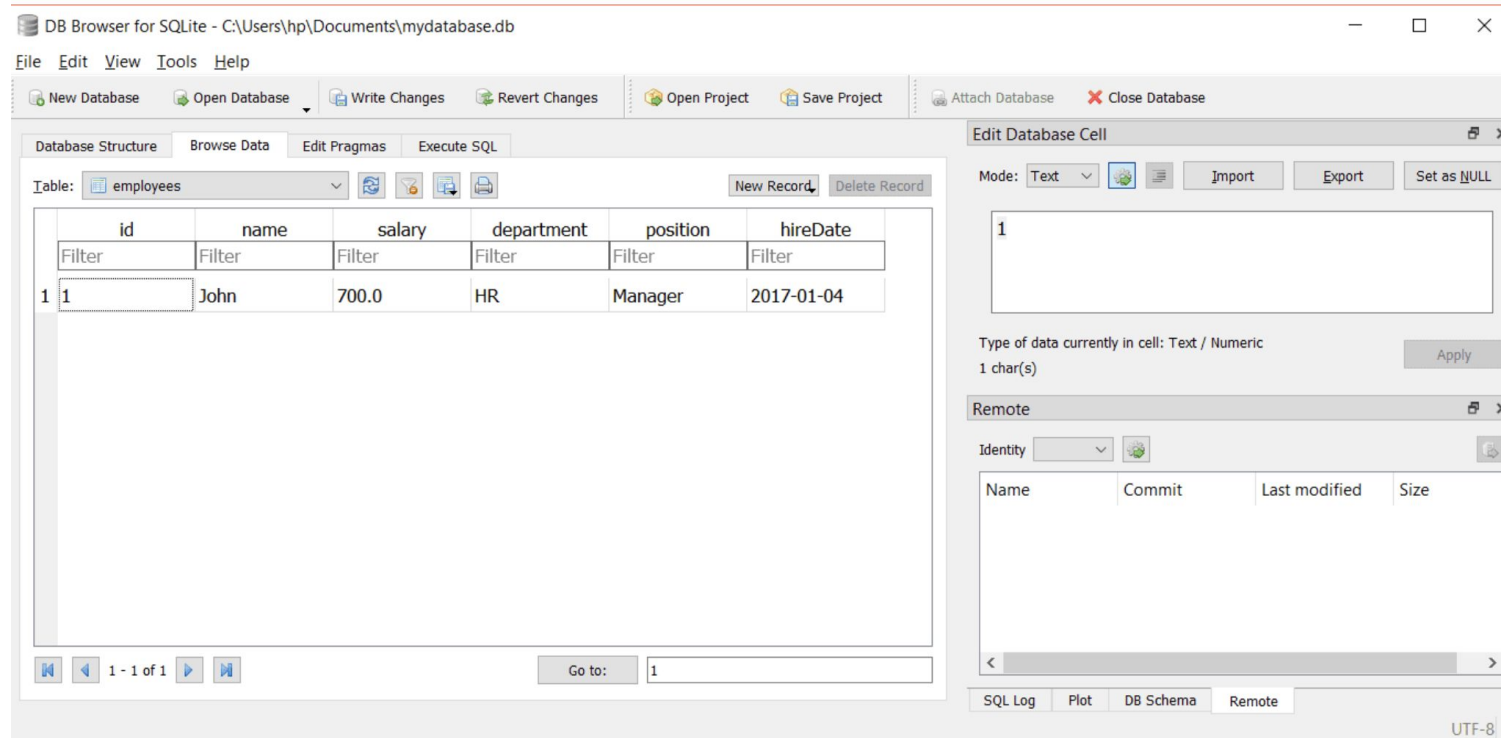
To check if our table is created, you can use the DB browser for SQLite to view your table. Open your mydatabase.db file with the program, and you should see your table:



Data insert

To insert data in a table, we use the INSERT INTO statement. Consider the following line of code:

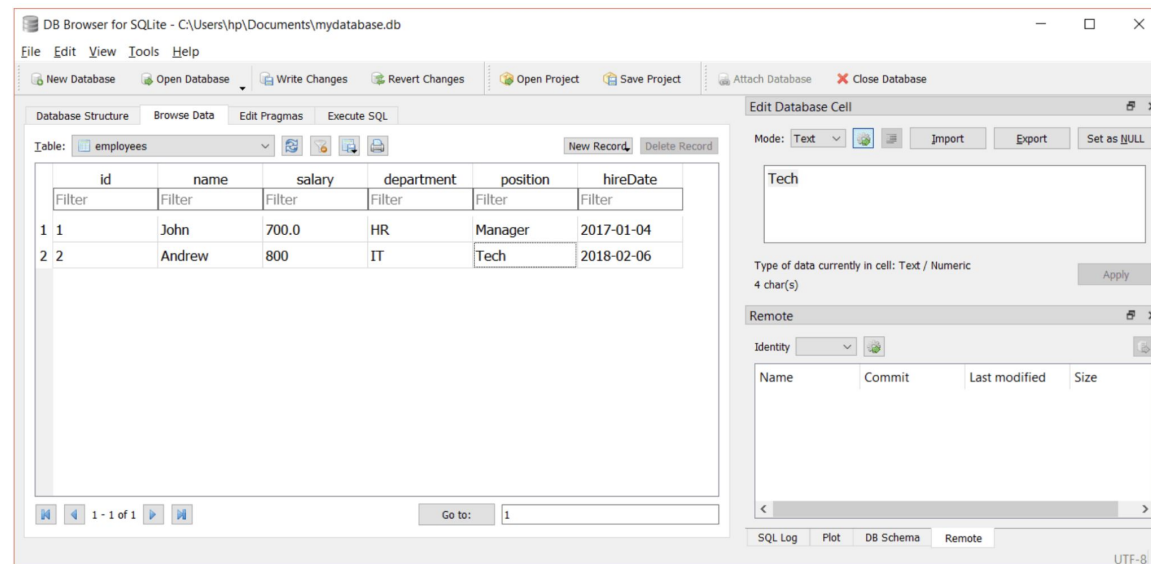
```
cursorObj = con.cursor()
cursorObj.execute("INSERT INTO employees VALUES(1, 'John', 700, 'HR', 'Manager', '2017-01-04')")
con.commit()
```



Data insert

We can also pass values/arguments to an INSERT statement in the execute() method. You can use the question mark (?) as a placeholder for each value. The syntax of the INSERT will be like the following:

```
sqlite_chapter.py
1 import sqlite3
2 con = sqlite3.connect('mydatabase.db')
3 def sql_insert(con, entities):
4     cursorObj = con.cursor()
5     cursorObj.execute('INSERT INTO employees(id, name, salary, department, position, hireDate) VALUES(?, ?, ?, ?, ?, ?)', entities)
6     con.commit()
7 entities = (2, 'Andrew', 800, 'IT', 'Tech', '2018-02-06')
8 sql_insert(con, entities)
```



Update table

- To update the table, simply create a connection, then create a cursor object using the connection and finally use the UPDATE statement in the execute() method.
- Suppose that we want to update the name of the employee whose id equals 2. For updating, we will use the UPDATE statement and for the employee whose id equals 2. We will use the WHERE clause as a condition to select this employee.

- Consider the following code:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
def sql_update(con):
    cursorObj = con.cursor()
    cursorObj.execute('UPDATE employees SET name = "Rogers" where id = 2')
    con.commit()
sql_update(con)
```

	id	name	salary	department	position	hireDate
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	John	700.0	HR	Manager	2017-01-04
2	2	Rogers	800.0	IT	Tech	2018-02-06

Select statement

- You can use the select statement to select data from a particular table. If you want to select all the columns of the data from a table, you can use the asterisk (*). The syntax for this will be as follows:

```
select * from table_name
```

- In SQLite3, the SELECT statement is executed in the execute method of the cursor object. For example, select all the columns of the employees' table, run the following code:

```
cursorObj.execute('SELECT * FROM employees ')
```

```
cursorObj.execute('SELECT id, name FROM employees')
```

Fetch data

- To fetch the data from a database, we will execute the SELECT statement and then will use the fetchall() method of the cursor object to store the values into a variable. After that, we will loop through the variable and print all values.
- The code will be like this:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
def sql_fetch(con):
    cursorObj = con.cursor()
    cursorObj.execute('SELECT * FROM employees')
    rows = cursorObj.fetchall()
    for row in rows:
        print(row)
sql_fetch(con)
```



Exercise

- Establish a connection with your root user.
- Create a new database called `second_db`.
- Create a table named `articles`.
- Create three fields to your database, `Author`, `name`, `date`.
- Add at least three different articles into your database, using the `executemany` command.

Flask & SQLITE

- Flask integration with SQLITE application



Application introduction

- We'll develop a Flask application that collects user input in a form and shows it on an additional web page using SQLite in Python.
- Please download Flask module: `pip install flask`
- We will create our three HTML pages and a Python Flask application that will communicate with the SQLITE database and will upload and display the data through the HTML files.

index.html

- The index.html file will contain two buttons, one button to check all the participant's lists (taken from the database). And the other button to create a new entry.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Flask and SQLite </title>
  </head>
  <body>
    <h1>Build Web App Using Flask and SQLite</h1>
    <button class="btn" type="button" onclick="window.location.href='{{ url_for('join') }}';">Fill form to get updates</button><br/>
    <button class="btn" type="button" onclick="window.location.href='{{ url_for('participants') }}';">Check participant list</button>
  </body>
</html>
```

join.html

- In the join.html, create a simple form that takes Name, Email, City, Country and Phone as the input to store in the database. By the POST method, receive the form request of all the columns and commit the changes in the database after inserting the details in the table.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Flask and SQLite </title>
  </head>
  <body>
    <form method="POST">
      <label>Enter Name:</label>
      <input type="name" name="name" placeholder="Enter your name" required><br/>
      <label>Enter Email:</label>
      <input type="email" name="email" placeholder="Enter your email" required><br/>
      <label>Enter City:</label>
      <input type="name" name="city" placeholder="Enter your City name" required><br/>
      <label>Enter Country:</label>
      <input type="name" name="country" placeholder="Enter the Country name" required><br/>
      <label>Enter phone num:</label>
      <input type="name" name="phone" placeholder="Your Phone Number" required><br/>
      <input type = "submit" value = "submit"/><br/>
    </form>
  </body>
</html>
```

participants.html

- Use table tag and assign the heading using <th> tag. To auto increment, the table row on the new entry, use a For loop jinja template. Inside For loop add <tr> and <td> tags.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Flask and SQLite </title>
5    </head>
6    <style>
7      table, th, td {
8        border:1px solid black;
9      }
10   </style>
11   <body>
12     <table style="width:100%">
13       <tr>
14         <th>Name</th>
15         <th>Email</th>
16         <th>City</th>
17         <th>Country</th>
18         <th>Phone Number</th>
19       </tr>
20       {%for participant in data%}
21         <tr>
22           <td>{{participant[0]}}</td>
23           <td>{{participant[1]}}</td>
24           <td>{{participant[2]}}</td>
25           <td>{{participant[3]}}</td>
```

```
26           <td>{{participant[4]}}</td>
27         </tr>
28       {%endfor%}
29     </table>
30   </body>
31 </html>
```

app.py introduction

- Create a new file named app.py and build a Front End of the Web App by rendering HTML templates. From here we shall go function by function explanation as in points:
- To insert the data into the database, we first need to create a new database table. The column to be inserted in the database is Name, Email, City, Country, and Phone Number.
- The basic syntax to start with sqlite3 is to first connect to the database. `sqlite3.connect("database.db")` will create a new database. The next step is to create a new table, but it will first check if the table already exists or not.
- One button in the index.html prompts to the participant's list, and thus using the existing database select * from the table and display it using a Python template i.e., Jinja template to run through the loop within HTML. In the following code, we have created a table tag, inside the table tag for every new insertion in the database, we add a Loop Jinja Template to auto increment the new table row.
- In the participants function, we use select all columns from the table name, we use `fetchall()` method you retrieve the data.

app.py code

```
1 from flask import Flask, render_template, request
2 import sqlite3
3
4 app = Flask(__name__)
5
6
7 @app.route('/')
8 @app.route('/home')
9 def index():
10     return render_template('index.html')
11
12
13 connect = sqlite3.connect('database.db')
14 connect.execute(
15     'CREATE TABLE IF NOT EXISTS PARTICIPANTS (name TEXT, \
16     email TEXT, city TEXT, country TEXT, phone TEXT)')
17
18
```

```
19 @app.route('/join', methods=['GET', 'POST'])
20 def join():
21     if request.method == 'POST':
22         name = request.form['name']
23         email = request.form['email']
24         city = request.form['city']
25         country = request.form['country']
26         phone = request.form['phone']
27
28         with sqlite3.connect("database.db") as users:
29             cursor = users.cursor()
30             cursor.execute("INSERT INTO PARTICIPANTS \
31             (name,email,city,country,phone) VALUES (?, ?, ?, ?, ?)",
32                             (name, email, city, country, phone))
33             users.commit()
34         return render_template("index.html")
35     else:
36         return render_template('join.html')
```

app.py code continue

```
39 @app.route('/participants')
40 def participants():
41     connect = sqlite3.connect('database.db')
42     cursor = connect.cursor()
43     cursor.execute('SELECT * FROM PARTICIPANTS')
44
45     data = cursor.fetchall()
46     return render_template("participants.html", data=data)
47
48
49 if __name__ == '__main__':
50     app.run(debug=False)
```

Build Web App Using Flask and SQLite

Fill form to get updates

Check participant list

Enter Name: Tarun R Jain

Enter Email: tarun@gmail.com

Enter City: Bengaluru

Enter Country: India

Enter phone num: 1111111111

submit

Enter Name: Rahul

Enter Email: rahul@gmail.com

Enter City: Bengaluru

Enter Country: India

Enter phone num: 0000000000

submit

Name	Email	City	Country	Phone Number
Tarun R Jain	tarun@gmail.com	Bengaluru	India	1111111111
Rahul	rahul@gmail.com	Bengaluru	India	0000000000

Flask SQLAlchemy

- Flask integration with SQL Alchemy



+



App introduction

- A Flask application that takes data in a form from the user and then displays it on another page on the website. We can also delete the data. We won't focus on the front-end part rather we will be just coding the backend for the web application.
- Modules that we need to install:
 - `pip install flask`
 - `pip install flask-sqlalchemy`
 - `pip install Flask-Migrate`
- In sqlalchemy we use classes to create our database structure. In our application, we will create a Profile table that will be responsible for holding the user's id, first name, last name, and age.
- The creation of the database and the table will be performed with the following command.
 - `flask db init`
 - `flask db migrate -m "Your message"`
 - `flask db upgrade`

index.html

```
2  <!DOCTYPE html>
3  <html>
4  <head>
5      <title>Index Page</title>
6  </head>
7  <body>
8      <h3>Profiles</h3>
9      <a href="/add_data">ADD</a>
10     <br>
11     <table>
12         <thead>
13             <th>Id</th>
14             <th>First Name</th>
15             <th>Last Name</th>
16             <th>Age</th>
17             <th>#</th>
18         </thead>
19         {% for data in profiles %}
20         <tbody>
21             <td>{{data.id}}</td>
22             <td>{{data.first_name}}</td>
23             <td>{{data.last_name}}</td>
24             <td>{{data.age}}</td>
25             <td><a href="/delete/{{data.id}}" type="button">Delete</a></td>
26         </tbody>
27         {% endfor %}
28     </table>
29 </body>
30 </html>
```

add_profile.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Add Profile</title>
5  </head>
6  <body>
7      <h3>Profile form</h3>
8      <form action="/add" method="POST">
9          <label>First Name</label>
10         <input type="text" name="first_name" placeholder="first name...">
11         <label>Last Name</label>
12         <input type="text" name="last_name" placeholder="last name...">
13         <label>Age</label>
14         <input type="number" name="age" placeholder="age..">
15         <button type="submit">Add</button>
16     </form>
17 </body>
18 </html>
```

app.py code

```
1 from flask import Flask, request, redirect
2 from flask.templating import render_template
3 from flask_sqlalchemy import SQLAlchemy
4 from flask_migrate import Migrate, migrate
5 app = Flask(__name__)
6 app.debug = True
7 # adding configuration for using a sqlite database
8 app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'
9 # Creating an SQLAlchemy instance
10 db = SQLAlchemy(app)
11 # Settings for migrations
12 migrate = Migrate(app, db)
13 # Models
14 class Profile(db.Model):
15     id = db.Column(db.Integer, primary_key=True)
16     first_name = db.Column(db.String(20), unique=False, nullable=False)
17     last_name = db.Column(db.String(20), unique=False, nullable=False)
18     age = db.Column(db.Integer, nullable=False)
19
20     def __repr__(self):
21         return f"Name : {self.first_name}, Age: {self.age}"
```

```
24 # function to render index page
25 @app.route('/')
26 def index():
27     profiles = Profile.query.all()
28     return render_template('index.html', profiles=profiles)
29
30
31 @app.route('/add_data')
32 def add_data():
33     return render_template('add_profile.html')
34
35
36 # function to add profiles
37 @app.route('/add', methods=["POST"])
38 def profile():
39     # In this function we will input data from the
40     # form page and store it in our database. Remember
41     # that inside the get the name should exactly be the same
42     # as that in the html input fields
43     first_name = request.form.get("first_name")
44     last_name = request.form.get("last_name")
45     age = request.form.get("age")
46
47     # create an object of the Profile class of models and
48     # store data as a row in our datatable
49     if first_name != '' and last_name != '' and age is not None:
50         p = Profile(first_name=first_name, last_name=last_name, age=age)
51         db.session.add(p)
52         db.session.commit()
53         return redirect('/')
54     else:
55         return redirect('/')

```

app.py code

```
58 @app.route('/delete/<int:id>')
59 def erase(id):
60     # deletes the data on the basis of unique id and
61     # directs to home page
62     data = Profile.query.get(id)
63     db.session.delete(data)
64     db.session.commit()
65     return redirect('/')
66
67
68 if __name__ == '__main__':
69     app.run()
```

Profiles

[ADD](#)

Id	First Name	Last Name	Age	#
1	Nikhil	Aggarwal	25	Delete

Profiles

[ADD](#)

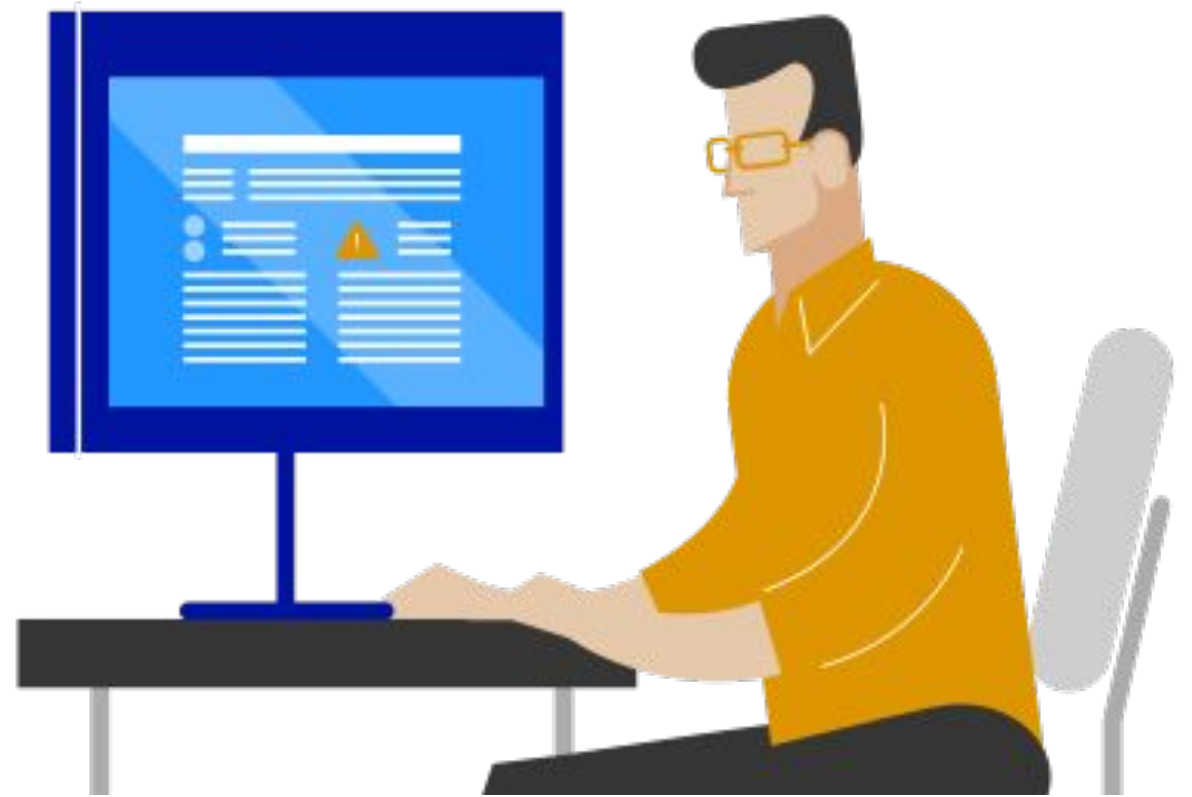
Id	First Name	Last Name	Age	#
----	------------	-----------	-----	---

Profile form

First Name Last Name Age

Aggarwal

Project - 2



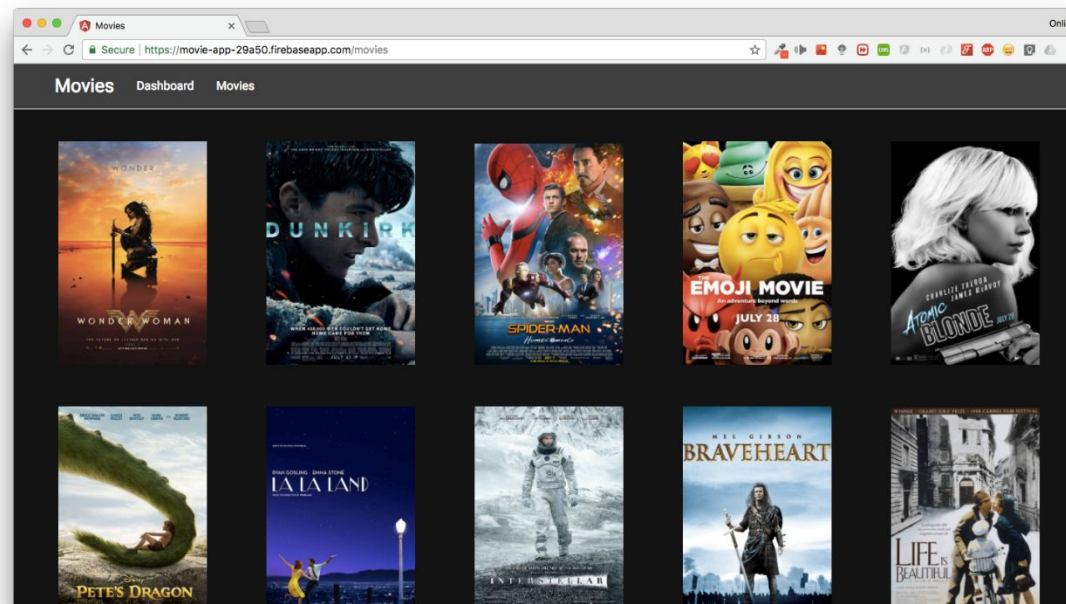
Project

- In this project, you will create a Flask application.
- You will create a database site for movies. Similar to the site of IMDB, but a little bit simpler.
- Your admin user will be in charge to post movies with the following data:
 - Poster with image of the movie
 - A title with the name of the movie.
 - A short description of the movie plot.
 - General information:
 - Director of the movie
 - Four main actors
 - Year of release



Home page

- On the homepage of your site, you are required to present just the images of movies.
- Something like that:



Movie description page

- By clicking on a movie image you will be moved to a page with the movie information.
 - Poster with image of the movie
 - A title with the name of the movie.
 - A short description of the movie plot.
 - General information:
 - Director of the movie
 - Four main actors
 - Year of release
 - The users of the site reviews



The design of the site

- You are free to use HTML files to design your site as you want.
- Please remember that you do must think about other user experience during the design of your site.
- Please try to use everything that we learned on Flask so far.



Thank you for your time

